

Caio Collino, Thiago Lins, Vinicius Vianna

Pesquisa - Computação Paralela

São Paulo

2026

Sumário

1	DESCRIÇÃO DOS ALGORITMOS	2
1.1	Implementação Sequencial	2
1.2	Implementação com o OpenMP	2
2	MEDIÇÃO DE TEMPO	3
3	DISCUSSÃO DOS RESULTADOS	4
4	CONCLUSÃO	6

1 Descrição dos Algoritmos

A sequência de fibonacci é uma sequência numérica onde cada termo é obtido pela soma dos dois termos anteriores. Seus casos bases são definidos da seguinte maneira:

$$fib(0) = 0$$

$$fib(1) = 1$$

Sua definição recursiva, para valores maiores ou iguais a 2, é:

$$fib(n) = fib(n - 1) + fib(n - 2)$$

Em sua implementação recursiva tradicional, o fibonacci chama a si mesmo duas vezes para cada um de seus valores de n maiores que 1, o quê acaba gerando uma grande quantia de chamadas, especificamente quando estamos lidando com valores maiores que a entrada.

1.1 Implementação Sequencial

O fibonacci sequencial foi implementado de maneira direta, onde todas as chamadas recursivas são executadas de uma vez por uma única thread. Para fazer a medição do tempo de execução da versão sequencial, utilizamos a função `clock()` da biblioteca `time.h`.

1.2 Implementação com o OpenMP

O fibonacci paralelo utiliza `#pragma omp task` para a criação de tarefas nas chamadas recursivas, nos permitindo que as chamadas `fibonacci_omp(n - 1)` e `fibonacci_omp(n - 2)` possam ser executadas paralelamente.

A *task* é uma diretiva que avisa ao OpenMP que aquele determinado trecho de código pode ser executado como uma tarefa independente. *taskwait* é outra diretiva que é usada com o fim de garantir que as duas tarefas sejam terminadas antes de ocorrer a soma dos resultados.

Definimos o seguinte limite sequencial:

```
#define LIMITE_SEQUENCIAL 25
```

O limite definido para evitar que tarefas sejam criadas para chamadas muito pequenas. Se cada chamada recursiva criasse tarefas novas, existiria um número muito grande de tarefas e o custo de gerenciamento iria sobrepor o ganho derivado do paralelismo.

A chamada principal da função paralela foi definida dentro de uma região paralela do OpenMP. A diretiva *parallel* cria o conjunto de threads, enquanto a *single* é responsável por garantir que apenas uma única *thread* inicie a execução da função recursiva principal. Com isso, torna-se possível a distribuição das tarefas entre as *threads* disponíveis.

2 Medição de Tempo

Para realizar a avaliação de desempenho, foi feita a comparação da versão sequencial e a versão paralela do algoritmo. Foi utilizado como entrada $n = 40$, já que esse valor já gera uma quantia significativa de chamadas recursivas, tornando possível a melhor observação das diferenças de tempo.

No fibonacci paralelo, o número de threads pode ser alterado através da variável `OMP_NUM_THREADS`. Foi utilizada a função `omp_get_wtime()` para a medição de tempo da versão paralela. Segue uma tabela dos resultados:

Tabela 1 – Comparação entre as versões sequencial e paralela

Versão	Entrada	Threads	Tempo aproximado
Sequencial	40	1	0,85 s
Paralela	40	2	0,55 s
Paralela	40	4	0,38 s
Paralela	40	8	0,35 s

Fonte: Elaborado pelo Autor

Os exemplos apresentados podem variar de acordo com o computador utilizador, processador, sistema operacional, compiladores e a quantidade de *threads* disponíveis.

Espera-se que haja uma melhora no desempenho na versão paralela quando existe trabalho suficiente para ser dividido entre threads. Esse ganho não aumenta indefinidamente com o aumento do número de threads.

3 Discussão dos Resultados

De acordo com os resultados, podemos concluir que a versão paralela pode apresentar ganhos de desempenho em comparação à versão sequencial, especialmente com valores maiores que n . O motivo para isso ocorrer é que o Fibonacci recursivo cria diversas chamadas independentes, o que torna possível o processamento de diferentes partes da árvore de recursão ao mesmo tempo por *threads* diferentes.

ainda assim, o desempenho da versão paralela não é definido apenas pelo número de *threads*. o *overhead* causado pela criação de tarefas é um dos principais influenciadores no resultado. Toda vez que uma diretiva `#pragma omp task` é executada, é necessário que o OpenMP realize a criação de uma nova tarefa, armazene-a internamente, gerencie-a e depois a distribua para uma *thread* disponível, esse processo tem um custo.

No Fibonacci, esse custo pode ser muito significativo, já que o número de chamadas recursivas cresce muito rapidamente. Se tarefas forem criadas até para valores menores que n , muitas delas terão baixa quantias de trabalho para realizar. nesse contexto específico, o tempo gasto na criação e no gerenciamento dessas tarefas podem ser maior que o tempo gasto executando o cálculo de maneira sequencial.

Devido a esse motivo, utilizar um limite sequencial é importante. Quando utilizamos a atribuição `limite_sequencial = 25`, evitamos que o programa crie tarefas para chamadas pequenas. isso garante a paralelização das partes maiores da árvore recursiva, ao mesmo tempo que as partes menores são resolvidas de maneira sequencial. isso reduz o *overhead* e melhora o desempenho em geral.

Temos como outro fator importante o balanceamento de carga. A árvore de chamadas do algoritmo de Fibonacci não é perfeitamente equilibrada: $fib(n - 1)$ geram mais trabalho que $fib(n - 2)$, já que ela cresce em uma quantia maior de chamadas recursivas. Isso implica que certas *threads* possam receber mais trabalho que as outras.

Em relação a esse ponto, o *work stealing* é muito importante. No contexto de uma execução paralela com tarefas, uma *thread* pode terminar suas tarefas antes de outras, e, quando isso acontece, pode buscar tarefas pendentes de outras *threads*. Isso proporciona uma melhor distribuição do trabalho e a redução do tempo de ociosidade de *threads*.

Ainda assim, a escalabilidade possui seus limites. Quando ocorre um aumento no número de *threads*, o tempo de execução tem a tendência de diminuir até certo ponto. Após isso, os ganhos passam a ser menos significativos. O motivo disso ocorrer é que o programa passa a ser prejudicado com custos de sincronização, gerenciamento de tarefas e disputa por recursos do processador.

Portanto, a utilização da versão paralela com OpenMP é benéfica quando existe uma quantia de trabalho suficiente para compensar o custo da criação de tarefas. Mas, quando lidando com entradas menores ou uma criação excessiva de tarefas, a versão sequencial pode ser igual ou

mais eficiente.

4 Conclusão

A realização dessa atividade tornou possível a observação de como é possível utilizar o OpenMP para paralelizar algoritmos recursivos através de tarefas.

A implementação sequencial do algoritmo serviu como uma base da comparação, enquanto a implementação paralela serviu para mostrar como as diretivas *task*, *taskwait*, *parallel* e *single* podem ser utilizadas em conjunto para distribuir o trabalho entre diversas *threads*.

De acordo com os resultados, o uso de tarefas recursivas pode trazer ganhos de desempenho, especificamente em cenários onde a entrada é grande o suficiente. Porém, também ficou evidente que o paralelismo não garante automaticamente uma performance superior. Existe um custo atribuído a criação de tarefas, e ele pode prejudicar o desempenho se o programa criar tarefas pequenas demais.

O uso de tarefas em OpenMP permite expressar paralelismo em algoritmos recursivos com facilidade. Ao invés de dividir o trabalho entre *threads* manualmente, é possível criar tarefas e permitir o OpenMP fazer o gerenciamento da distribuição. Também, mecanismos como o *work stealing* podem apresentar uma melhora no balanceamento de carga, especialmente no contexto de algoritmos com árvores de recursão desbalanceadas.

Temos como principal limitação o *overhead*. A criação de muitas tarefas pequenas pode resultar em uma versão paralela menos eficiente. Considerando isso, estratégias como a utilização de um limite sequencial são importantes para a obtenção de bons resultados.

Concluimos que a utilização de OpenMP com *task* é uma ferramenta útil para a paralelização de algoritmos recursivos, assumindo que cuidados sejam tomados. O desempenho final não é definido apenas por número de *threads*, mas também pela maneira de como as tarefas são criadas, do tamanho da entrada, do balanceamento de carga e da redução do *overhead*.